

JAVA EE 8 FRONT-END - ANGULAR (V19)

Módulo 2: Componentes y Control de Flujo Moderno

Introducción al Módulo

La ingeniería de software en frontend empresarial exige arquitecturas orientadas a componentes autónomos de alto rendimiento, bajo acoplamiento y legibilidad declarativa. En este módulo, consolidarás el modelo de Componentes Standalone (ver ANEXO I), superando de forma definitiva la sobrecarga de los módulos clásicos. A través de la adopción del nuevo flujo de control nativo del compilador (@if, @for, @switch) y la técnica de renderizado diferido con @defer (ver ANEXO I y ANEXO III), dominarás las herramientas esenciales para reducir el tamaño de los paquetes iniciales de distribución, optimizar la experiencia de usuario y estructurar paneles reactivos de nivel corporativo.

Estructura Temática y Hoja de Ruta Formativa

Este manual técnico está concebido como una guía teórico-práctica secuencial articulada en los siguientes puntos principales:

- **1 Creación de Componentes Standalone:** Eliminación de NgModules, metadatos autónomos y directrices de importación selectiva.
- **2 Data Binding: Property, Event y Two-Way:** Comunicación unidireccional por propiedades y eventos, contexto histórico de FormsModule/[ngModel] y enlace bidireccional moderno en Angular 19 mediante Signals y la primitiva model().
- **3 El Nuevo Flujo de Control Declarativo:** Sintaxis integrada en el compilador (@if, @for y @switch), seguimiento obligatorio con track y bloque @empty.
- **4 Uso de @defer para Carga Diferida (Lazy Rendering):** Arquitectura de sub-bloques (@placeholder, @loading, @error) y orquestación de disparadores (triggers) declarativos.
- **5 Ejercicio Final:** Proyecto integrador de panel de administración empresarial optimizado con reactividad granular de Signals y carga diferida con @defer.
- **6 Resumen Ejecutivo del Conocimiento Adquirido:** Síntesis formal de competencias, directrices de arquitectura y patrones de ingeniería.

Anexos Técnicos de Referencia:

- **ANEXO I (Glosario de Términos Técnicos):** Compendio conceptual con la definición de términos clave citados con (ver ANEXO I).
- **ANEXO II (Guía de Migración al Nuevo Control de Flujo):** Tabla comparativa de directivas clásicas frente a sintaxis de compilador y comandos de CLI, citado con (ver ANEXO II).
- **ANEXO III (Catálogo Técnico de Disparadores @defer):** Matriz de triggers declarativos y patrones de precarga en reposo, citado con (ver ANEXO III).
- **ANEXO IV (¿Qué es un JSON?):** Estándar de intercambio de datos, tipología, serialización y validación estructural, citado con (ver ANEXO IV).
- **ANEXO V (Guía de Scaffolding en Angular):** Andamiaje de proyectos, mapa de archivos de configuración y arquitectura por capas empresariales, citado con (ver ANEXO V).
- **ANEXO VI (Arquitectura Standalone en Angular v19):** Estándar nativo autónomo, anatomía de componentes limpios y optimización de compilación, citado con (ver ANEXO VI).

1 Creación de Componentes Standalone

1.1 El Cambio de Paradigma: Adiós a los NgModules

Históricamente, Angular dependía de los módulos (@NgModule) para declarar la existencia de componentes, directivas y tuberías, así como para gestionar la inyección de dependencias y la visibilidad de elementos entre distintos módulos. Este enfoque generaba:

- **Acoplamiento indirecto:** Dificultad para seguir el origen de un componente o directiva importada.
- **Sobrecarga cognitiva:** Excesivo código base (*boilerplate*) para la creación de un simple elemento visual.
- **Complejidad en Lazy Loading:** Curva de aprendizaje empinada para estructurar cargas perezosas mediante subrutinas ligadas a módulos secundarios.

Los Componentes Standalone (introducidos de forma estable en Angular 15 y consolidados como el estándar nativo por defecto en Angular 19) resuelven esta problemática permitiendo que cada componente gestione sus propias dependencias directamente desde su metadato @Component.

1.2 Estructura y Metadatos de un Componente Standalone

Para declarar un componente como autónomo, se añade la propiedad `standalone: true` dentro de su decorador, y se listan las directivas, pipes o subcomponentes que requiere internamente dentro de su propia matriz de `imports`:

```
import { Component } from '@angular/core';
import { MatButtonModule } from '@angular/material/button';

@Component({
  selector: 'app-user-profile',
  standalone: true,
  imports: [MatButtonModule],
  template: `
    <div class="profile-card">
      <h2>Perfil del Usuario</h2>
      <button mat-raised-button color="primary">Ver Detalle</button>
    </div>
  `,
  styles: [
    .profile-card { padding: 20px; border: 1px solid #ccc; border-radius: 8px; }
  ]
})
export class UserProfileComponent {
  // Lógica del componente
}
```

Listing 1: p1-p1-2-user-profile.component.ts

Nota Didáctica: Importación Selectiva y Rendimiento

Al migrar a componentes standalone en Angular 19, ya no es necesario importar `CommonModule`. Al adoptar el nuevo flujo de control, el renderizado condicional, la iteración y las estructuras de selección se gestionan directamente a nivel de compilador sin requerir módulos de directivas auxiliares. Esto maximiza la efectividad del proceso de *tree-shaking* (ver ANEXO I) y aligera sustancialmente el peso final de los paquetes distribuidos.

Miniejercicio: Declaración Atómica de Componente Standalone

Enunciado: Desarrolla un componente autónomo mínimo denominado `BadgeEstadoComponent`

que declare su selector, marque `standalone: true` y renderice una etiqueta informativa con estilos encapsulados sin importar ningún módulo externo.

Solución:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-badge-estado',
  standalone: true,
  template: `
    <span class="badge-online">Nodo Operativo</span>
  `,
  styles: [`
    .badge-online { background: #F0FDF4; color: #059669; border: 1px solid #86EFAC; padding: 3px
    ↪ 8px; border-radius: 4px; font-weight: bold; font-size: 0.8rem; }
  `]
})
export class BadgeEstadoComponent {}
```

Listing 2: p1-mini-badge-estado.component.ts

Explicación:

- **Aislamiento y ligereza:** Al marcar `standalone: true` y omitir importaciones innecesarias, el componente resulta autocontenido, ligero y óptimo para su empaquetado directo con *tree-shaking*.

2 Data Binding: Property, Event y Two-Way

El flujo de comunicación entre la lógica TypeScript y la interfaz gráfica HTML es fundamental en cualquier framework moderno. Angular divide este intercambio en tres estrategias principales perfectamente integradas con la reactividad nativa.

2.1 Property Binding ([propiedad] = "expresión")

Establece la comunicación de una sola vía (unidireccional) desde el modelo lógico hacia la propiedad del elemento del DOM o una propiedad de entrada (`@Input` o la primitiva `input()` basada en señales) de un componente secundario.

```
// Componente TypeScript
imageUrl = 'assets/logos/it-formacion.png';
isDisabled = true;

// Template HTML
<img [src]="imageUrl" alt="Logo corporativo">
<button [disabled]="isDisabled">Enviar Solicitud</button>
```

Listing 3: p2-p2-1-property-binding.html

2.2 Event Binding ((evento) = "método()")

Permite escuchar las interacciones del usuario en el DOM (clicks, pulsaciones de teclado, movimientos de ratón) o capturar eventos personalizados emitidos por componentes hijos (`@Output` o la función reactiva `output()`) enviando la información hacia la lógica de TypeScript.

```
// Componente TypeScript
onSave(event: MouseEvent) {
  console.log('Botón pulsado', event);
}
```

```

}

// Template HTML
<button (click)="onSave($event)">Guardar Cambios</button>

```

Listing 4: p2-p2-2-event-binding.html

2.3 Two-Way Binding y su Contexto Histórico en Angular 19

El enlace bidireccional sincroniza el modelo con la interfaz visual.

Contexto Histórico (El Enfoque Clásico con FormsModule):

En versiones clásicas de Angular, la sincronización de campos dependía del paquete externo `FormsModule` y la directiva `[(ngModel)]` (la célebre sintaxis *banana-in-a-box*). Este mecanismo dependía estrechamente de `Zone.js` para capturar mutaciones globales, requiriendo módulos adicionales e incurriendo en sobrecarga de empaquetado. En proyectos modernos basados en Angular 19 **se desincentiva por completo el uso de `[(ngModel)]`**, considerándose un patrón puramente histórico o legado.

Estándar Moderno en Angular 19: Signals y la Primitiva `model()`:

En Angular 19, el enlace bidireccional entre componentes se implementa mediante la API reactiva `model()`. Para elementos del DOM nativo, la reactividad granular de `signal()` se vincula limpiamente a través de `Property Binding` en la lectura y `Event Binding` en la escritura, evitando dependencias externas:

```

import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-text-input',
  standalone: true,
  template: `
    <div class="input-container">
      <input
        [value]="username()"
        (input)="username.set($any($event.target).value)"
        placeholder="Escribe tu nombre">
      <p>El nombre de usuario actual es: {{ username() }}</p>
    </div>
  `
})
export class TextInputComponent {
  username = signal<string>('');
}

```

Listing 5: p2-p2-3-text-input.component.ts

Miniejercicio: Sincronización Reactiva de Filtro con Signals

Enunciado: Construye un campo de texto interactivo en Angular 19 que sincronice una señal reactiva sin utilizar `FormsModule` ni `ngModel`, proveyendo un botón para restablecer el término a una cadena vacía.

Solución:

```

import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-filtro-busqueda',
  standalone: true,

```

```

template: `
  <input
    [value]="termino()"
    (input)="actualizarTermino($event)"
    placeholder="Buscar servicio...">
  <button (click)="limpiar()">Restablecer</button>
  <span>Filtro actual: {{ termino() }}</span>
`
})
export class FiltroBusquedaComponent {
  termino = signal<string>('');

  actualizarTermino(event: Event) {
    const input = event.target as HTMLInputElement;
    this.termino.set(input.value);
  }

  limpiar() {
    this.termino.set('');
  }
}

```

Listing 6: p2-mini-filtro-busqueda.component.ts

Explicación:

- **Reactividad Granular Sin NgModel:** Mantiene el estado en una señal `signal<string>('')`. La lectura se enlaza directamente a `[value]` y la mutación se efectúa mediante `.set()`, logrando sincronización de precisión sin sobrecarga de directivas obsoletas.

3 El Nuevo Flujo de Control Declarativo

Angular 19 consolida una sintaxis nativa integrada a nivel de compilador. Este modelo suprime las dependencias de directivas auxiliares en la vista, reduce sustancialmente el coste de análisis estático del template, agiliza la verificación estricta de tipos y proporciona un rendimiento de renderizado superior con menor peso de compilación.

3.1 Renderizado Condicional con @if

La sintaxis declarativa de `@if` unifica el flujo condicional dentro de la plantilla con máxima legibilidad y sin requerir contenedores auxiliares. Permite encadenar bloques alternativos `@else if` y `@else` de forma fluida y natural:

```

@if (isLoggedIn) {
  <div class="user-card">
    <p>Bienvenido, {{ user.name }}</p>
  </div>
} @else if (isGuest) {
  <div class="guest-card">
    <p>Bienvenido, Invitado. Modo de exploración activo.</p>
  </div>
} @else {
  <div class="anon-card">
    <p>Por favor, inicia sesión en la plataforma para acceder a tus servicios.</p>
  </div>
}

```

Listing 7: p3-p3-1-condicional-declarativo.html

3.2 Iteración Estructurada y Optimizada con @for

El renderizado de listas con @for se ha optimizado por completo. La propiedad de seguimiento (track) es ahora ****obligatoria****. El compilador de Angular utiliza esta clave para minimizar la recreación de elementos del DOM cuando la lista cambia, eliminando los clásicos cuellos de botella de rendimiento asociados a trackBy cuando olvidabas implementarlo.

Además, incorpora el bloque nativo @empty, que se renderiza automáticamente si la lista de datos está vacía o nula.

```
@for (course of courses; track course.id) {  
  <div class="course-item">  
    <h3>{{ course.title }}</h3>  
    <p>Duración: {{ course.hours }} horas</p>  
  </div>  
} @empty {  
  <p class="no-data">No hay cursos disponibles actualmente.</p>  
}
```

Listing 8: p3-p3-2-iteracion-for-moderna.html

Dentro del bloque @for, tienes acceso inmediato a variables de contexto muy útiles de forma implícita:

- \$index: El índice base cero del elemento actual.
- \$first: Booleano que indica si es el primer elemento de la lista.
- \$last: Booleano que indica si es el último elemento de la colección.
- \$count: El tamaño total de la colección iterada.

3.3 Selección Múltiple con @switch

El renderizado condicional de múltiples casos reduce su sintaxis a un bloque similar al operador de control de flujo nativo de JavaScript:

```
@switch (userRole) {  
  @case ('admin') {  
    <app-admin-panel />  
  }  
  @case ('editor') {  
    <app-editor-panel />  
  }  
  @default {  
    <app-reader-panel />  
  }  
}
```

Listing 9: p3-p3-3-switch-declarativo.html

Miniejercicio: Iteración Indexada con Variables de Contexto

Enunciado: Renderiza una lista de direcciones IP utilizando el bloque @for, mostrando el índice contextual \$index y aplicando una etiqueta especial al primer elemento con \$first.

Solución:

```
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-lista-ips',  
  standalone: true,
```

```

template: `
  @for (ip of ips; track ip) {
    <div class="fila-ip">
      <span>#{ $index + 1 } - { ip }</span>
      @if ($first) { <strong class="badge-primary">[Gateway Principal]</strong> }
    </div>
  } @empty {
    <p>No se registran direcciones asignadas.</p>
  }
`
})
export class ListaIpsComponent {
  ips: string[] = ['192.168.1.1', '192.168.1.10', '192.168.1.25'];
}

```

Listing 10: p3-mini-lista-ips.component.ts

Explicación:

- **Contexto implícito:** Las variables `$index` y `$first` son provistas por el compilador en cada iteración sin declarar alias locales adicionales.

4 Uso de @defer para Carga Diferida (Lazy Rendering)

La directiva nativa `@defer` es una de las innovaciones de rendimiento más impresionantes integradas en los frameworks de frontend modernos. Permite retrasar la carga y el renderizado de un componente pesado de la aplicación hasta que se cumplan condiciones o disparadores lógicos específicos del cliente.

4.1 Sub-bloques Arquitectónicos de @defer

Un bloque diferido se define con hasta cuatro sub-bloques condicionales que ayudan a mitigar los efectos visuales de la transición de red:

1. **@defer:** Bloque principal que contiene el componente a cargar de forma diferida.
2. **@placeholder:** Contenido alternativo que se muestra por defecto en la pantalla inicial antes de que se dispare la carga. Puedes parametrizar un tiempo mínimo de visualización (`minimum 500ms`) para evitar parpadeos bruscos.
3. **@loading:** Contenido a mostrar en el intervalo en el que los bundles JS diferidos están siendo descargados desde el servidor.
4. **@error:** Contenido que se mostrará en pantalla en caso de fallo de red o error de resolución de carga.

4.2 Tipos de Disparadores (Triggers)

Puedes mezclar dos tipos principales de disparadores: `on` (disparadores basados en eventos del usuario o del sistema) y `when` (expresiones lógicas personalizadas evaluadas en TypeScript).

```

@defer (on viewport; prefetch on idle) {
  <!-- Componente pesado que será descargado asincrónicamente -->
  <app-heavy-charts [data]="reportData" />
} @placeholder {
  <div class="skeleton">Cargando previsualización... (Haz scroll para cargar el gráfico real)</div>
} @loading {
  <p>Cargando recursos...</p>
} @error {

```

Disparador	Descripción Operativa
on idle	Disparador por defecto. Carga el fragmento cuando el hilo principal del navegador está inactivo.
on viewport	Comienza la descarga cuando el área marcada entra en el campo de visión del usuario (útil para scroll infinito o contenido <i>below-the-fold</i>).
on interaction	Se descarga el bundle cuando haces clic o te enfocas en el área del placeholder.
on hover	Carga el contenido cuando el cursor del ratón pasa por encima del elemento.
on timer(x)	Retrasa la carga del bundle una cantidad de tiempo exacta (ej. <code>timer(500ms)</code>).
when condición	Disparo gobernado por una condición lógica booleana evaluada en TypeScript.

```
<p class="error-msg">Fallo al descargar el gráfico. Reintenta de nuevo más tarde.</p>
}
```

Listing 11: p4-p4-2-defer-interactivo.html

¡Atención en Desarrollo!

Para que un componente sea elegible por `@defer` para lazy loading, dicho componente **debe ser standalone** y no estar acoplado ni importado estáticamente por otro módulo principal fuera de un bloque diferido.

Miniejercicio: Carga Diferida Activada por Cursor (on hover)

Enunciado: Implementa un bloque `@defer` que postergue la carga de un componente de auditoría hasta que sitúes el ratón sobre un contenedor de previsualización (`on hover`).

Solución:

```
@defer (on hover) {
  <app-auditoria-detalle />
} @placeholder {
  <div class="hover-box">Sitúa el cursor aquí para cargar el panel de auditoría</div>
} @loading (minimum 200ms) {
  <p class="cargando">Descargando registros...</p>
}
```

Listing 12: p4-mini-defer-hover.html

Explicación:

- **Ahorro de ancho de banda:** La directiva `on hover` retiene la transferencia del script hasta que sobrevuelas el *placeholder*, evitando descargar código innecesario.

5 5 Ejercicio Final

Panel de Control y Monitoreo de Proyectos con Carga Diferida**Enunciado**

Construye un panel de gestión técnica en un componente standalone que integre:

1. Captura y sincronización de datos con Signals nativos (`signal()`) para ingresar proyectos especificando su nombre y criticidad (*baja, media, alta*), prescindiendo por completo de `FormsModule` o `[(ngModel)]`.

2. Una sección de estadísticas dinámica con @if que reporte la cantidad de proyectos activos o muestre un aviso si no existen elementos.
3. Un listado ordenado con @for que aproveche las variables de contexto \$index y gestione la lista vacía mediante @empty.
4. Una sección de auditoría avanzada que cargue un componente de telemetría de forma diferida mediante @defer (on interaction; prefetch on idle), incorporando su correspondiente @placeholder, @loading y @error.

Solución

```
import { Component, signal } from '@angular/core';

interface Proyecto {
  id: number;
  nombre: string;
  prioridad: 'baja' | 'media' | 'alta';
  completado: boolean;
}

@Component({
  selector: 'app-mini-dashboard',
  standalone: true,
  template: `
    <div class="dashboard-container">
      <header class="db-header">
        <h2>Panel de Control Técnico</h2>
        <span class="badge-version">Angular 19 Standalone</span>
      </header>

      <!-- Sección de Formulario Reactivo con Signals -->
      <section class="form-section">
        <h3>Agregar Nuevo Proyecto</h3>
        <div class="input-group">
          <input
            [value]="nuevoNombre()"
            (input)="nuevoNombre.set($any($event.target).value)"
            placeholder="Nombre del proyecto..." />
          <select
            [value]="nuevaPrioridad()"
            (change)="nuevaPrioridad.set($any($event.target).value)">
            <option value="baja">Baja Prioridad</option>
            <option value="media">Media Prioridad</option>
            <option value="alta">Prioridad Crítica</option>
          </select>
          <button (click)="agregarProyecto()">Insertar</button>
        </div>
      </section>

      <!-- Estadísticas Dinámicas con @if -->
      <section class="stats-section">
        @if (listaProyectos().length > 0) {
          <p>Total de proyectos activos: <strong>{{ listaProyectos().length }}</strong></p>
        } @else {
          <div class="alert warn">Ningún proyecto listado. Comienza rellenando el formulario.</div>
        }
      </section>
    </div>`
})
```

```

    }
  </section>

  <!-- Listado con @for e índices de contexto -->
  <section class="list-section">
    <h3>Lista de Control</h3>
    @for (proy of listaProyectos(); track proy.id) {
      <div class="proyecto-item" [class.alta-prioridad]="proy.prioridad === 'alta'">
        <div class="info">
          <span class="index">#{{ $index + 1 }}</span>
          <span class="nombre">{{ proy.nombre }}</span>
          <span class="tag" [class]="proy.prioridad">{{ proy.prioridad.toUpperCase() }}</
    ↪ span>
        </div>
        <div class="actions">
          <button (click)="eliminarProyecto(proy.id)">Eliminar</button>
        </div>
      </div>
    } @empty {
      <div class="empty-state">
        <p>La lista de proyectos se encuentra limpia en este momento.</p>
      </div>
    }
  </section>

  <!-- Carga Diferida con @defer -->
  <section class="defer-section">
    @defer (on interaction; prefetch on idle) {
      <div class="telemetria-box">
        <h4>Módulo de Auditoría y Telemetría</h4>
        <p>Datos analíticos cargados diferidamente bajo demanda del operador.</p>
      </div>
    } @placeholder (minimum 300ms) {
      <button class="btn-defer">Haga clic para inicializar la consola de auditoría</button>
    } @loading {
      <p class="loading-text">Descargando módulo de telemetría...</p>
    } @error {
      <p class="error-text">Fallo de conexión al transferir el artefacto analítico.</p>
    }
  </section>
</div>
`,
  styles: [`
.dashboard-container { font-family: 'Noto Sans', sans-serif; max-width: 650px; margin: 0
    ↪ auto; padding: 20px; border: 1px solid #CBD5E1; border-radius: 8px; background: #ffffff;
    ↪ }
.db-header { display: flex; justify-content: space-between; align-items: center; border-
    ↪ bottom: 2px solid #1E75B8; padding-bottom: 10px; margin-bottom: 20px; }
.db-header h2 { color: #1F2758; margin: 0; font-size: 1.3rem; }
.badge-version { background: #1F2758; color: white; padding: 2px 8px; border-radius: 12px;
    ↪ font-size: 0.75rem; font-weight: bold; }
.form-section { background: #F8FBFE; padding: 15px; border-radius: 6px; margin-bottom: 20px;
    ↪ border: 1px solid #E2E8F0; }
.input-group { display: flex; gap: 10px; }
.input-group input { flex: 2; padding: 8px; border: 1px solid #ccc; border-radius: 4px; }
.input-group select { flex: 1; padding: 8px; border: 1px solid #ccc; border-radius: 4px; }

```

```

.input-group button { background: #1E75B8; color: #fff; border: none; padding: 8px 15px;
↵ border-radius: 4px; cursor: pointer; font-weight: bold; }
.proyecto-item { display: flex; justify-content: space-between; align-items: center; padding
↵ : 10px; border-bottom: 1px solid #eee; }
.proyecto-item.alta-prioridad { background-color: #FEF2F2; border-left: 4px solid #DC2626; }
.info { display: flex; align-items: center; gap: 10px; }
.index { color: #95a5a6; font-weight: bold; }
.tag { font-size: 11px; padding: 2px 6px; border-radius: 3px; font-weight: bold; }
.tag.alta { background: #DC2626; color: white; }
.tag.media { background: #D97706; color: white; }
.tag.baja { background: #059669; color: white; }
.empty-state { text-align: center; color: #7f8c8d; padding: 20px; border: 2px dashed #CBD5E1
↵ ; border-radius: 6px; }
.defer-section { margin-top: 20px; padding-top: 15px; border-top: 1px solid #E2E8F0; }
.btn-defer { width: 100%; padding: 10px; background: #F8FBFE; border: 1px dashed #1E75B8;
↵ color: #1E75B8; font-weight: bold; border-radius: 6px; cursor: pointer; }
.telemetria-box { background: #F0FDF4; border: 1px solid #86EFAC; padding: 12px; border-
↵ radius: 6px; color: #065F46; }
.loading-text { color: #1E75B8; text-align: center; font-style: italic; }
.error-text { color: #DC2626; text-align: center; }
`]
})
export class MiniDashboardComponent {
  listaProyectos = signal<Proyecto[]>([
    { id: 1, nombre: 'Migración a Angular 19', prioridad: 'alta', completado: false },
    { id: 2, nombre: 'Refactorización de módulo Auth', prioridad: 'media', completado: true }
  ]);
  nuevoNombre = signal<string>('');
  nuevaPrioridad = signal<'baja' | 'media' | 'alta'>('baja');

  agregarProyecto() {
    const nombre = this.nuevoNombre().trim();
    if (!nombre) return;
    const nuevo: Proyecto = {
      id: Date.now(),
      nombre,
      prioridad: this.nuevaPrioridad(),
      completado: false
    };
    this.listaProyectos.update(proyectos => [...proyectos, nuevo]);
    this.nuevoNombre.set('');
    this.nuevaPrioridad.set('baja');
  }

  eliminarProyecto(id: number) {
    this.listaProyectos.update(proyectos => proyectos.filter(p => p.id !== id));
  }
}

```

Listing 13: p5-dashboard.component.ts

Explicación

- **Carga Asíncrona con @defer:** El componente de auditoría permanece aislado del bundle inicial y solo se recupera de la red cuando haces clic sobre el botón (on interaction), aprovechando los ciclos ociosos para precargarlo (prefetch on idle).

- **Control de Flujo Unificado:** La combinación de `@if`, `@for` con `track` y `@empty` simplifica drásticamente la estructura de la plantilla sin depender de directivas estructurales tradicionales.
- **Arquitectura Pura con Signals:** Se elimina por completo el uso de `[(ngModel)]` y `FormsModule`. La sincronización de formularios se gobierna mediante primitivas reactivas `signal()` conectadas a eventos del DOM nativo con granularidad de actualización $\mathcal{O}(k)$.

6 Resumen Ejecutivo del Conocimiento Adquirido

La consolidación de los componentes autónomos y del nuevo flujo de control nativo en Angular 19 establece un estándar superior de arquitectura frontend:

- **Arquitectura Standalone por Defecto:** Erradica la complejidad y sobrecarga de los módulos clásicos (`@NgModule`), agilizando el aislamiento de componentes y optimizando el empaquetado final mediante *tree-shaking* (ver ANEXO I).
- **Control de Flujo Nativo del Compilador:** La sintaxis integrada (`@if`, `@for`, `@switch`) supera sustancialmente las micro-sintaxis directivas clásicas (`*ngIf`, `*ngFor`), ofreciendo una verificación de tipos más estricta y obligando al uso de la clave `track` para evitar cuellos de botella en el renderizado del DOM (ver ANEXO I).
- **Vistas Diferidas con `@defer`:** Permite a los equipos de desarrollo implementar de forma declarativa patrones sofisticados de carga perezosa a nivel de vista, mitigando el tamaño del *bundle* inicial y mejorando sensiblemente las métricas de rendimiento web (ver ANEXO III).

ANEXO I: Glosario de Términos Técnicos

1. **Standalone Component:** Unidad gráfica autónoma en Angular que declara sus propias dependencias en el decorador `@Component`, prescindiendo de la intermediación de módulos tradicionales `@NgModule`.
2. **Control Flow Syntax:** Sintaxis nativa introducida a nivel de compilador en Angular que reemplaza las directivas estructurales tradicionales (`*ngIf`, `*ngFor`, `ngSwitch`) por bloques integrados con prefijo arroba (`@if`, `@for`, `@switch`).
3. **Track Expression:** Expresión obligatoria en el bloque `@for` que provee una clave única de seguimiento al compilador para correlacionar los elementos de una colección con sus respectivos nodos en el DOM.
4. **Empty Block (@empty):** Sub-bloque integrado de la directiva `@for` que se renderiza automáticamente cuando la colección iterada no contiene elementos o es nula.
5. **Deferrable Views (@defer):** Tecnología de carga diferida en tiempo de ejecución que retrasa la descarga y compilación de componentes de la vista hasta que se cumplan disparadores (*triggers*) o condiciones específicas en el cliente.
6. **Placeholder Block (@placeholder):** Sección visual que se muestra inmediatamente antes de que se active el disparador de un bloque diferido, admitiendo un tiempo mínimo de visualización (*minimum*).
7. **Two-Way Binding Moderno:** Patrón de sincronización de doble vía que en Angular 19 se articula mediante la primitiva reactiva `model()` o mediante señales de lectura y eventos de mutación nativos, relegando `[(ngModel)]` a un contexto histórico previo a la reactividad por Signals.
8. **Tree-Shaking:** Proceso automático de optimización del compilador orientado a eliminar código muerto o dependencias no referenciadas del paquete final distribuible.
9. **Core Web Vitals:** Conjunto de métricas estandarizadas por Google que evalúan la experiencia de usuario en términos de velocidad de carga, interactividad y estabilidad visual.

ANEXO II: Guía de Migración al Nuevo Control de Flujo

Para modernizar proyectos que emplean directivas estructurales tradicionales hacia la sintaxis integrada de Angular 19, debes considerar las siguientes equivalencias técnicas:

Sintaxis Clásica (Directiva)	Sintaxis Moderna (Compilador)
<code>*ngIf="condicion; else fallback"</code>	<code>@if (condicion) { ... } @else { ... }</code>
<code>*ngFor="let item of lista; trackBy: fn"</code>	<code>@for (item of lista; track item.id) { ... } @empty { ... }</code>
<code>[ngSwitch]="valor"</code>	<code>@switch (valor) { @case (v1) { ... } @default { ... } }</code>

1 Comando Automatizado de Migración CLI

Angular CLI provee un comando de migración asistida que analiza las plantillas HTML del proyecto y refactoriza automáticamente las directivas clásicas a bloques de compilador:

```
ng generate @angular/core:control-flow
```

ANEXO III: Catálogo Técnico de Disparadores @defer

El uso de @defer permite componer disparadores múltiples combinando la condición de carga con la precarga anticipada en reposo:

Disparador Declarativo	Caso de Uso Recomendado en Arquitectura Frontend
<code>on viewport</code>	Componentes extensos situados por debajo del área visible inicial (<i>below-the-fold</i>), como tablas de registros históricos o mapas interactivos.
<code>on interaction</code>	Secciones que exigen una acción deliberada previa del operador, como paneles desplegados, modales de configuración o visores de telemetría.
<code>on hover</code>	Elementos informativos emergentes (<i>tooltips</i> pesados) o previsualizaciones contextuales de perfil.
<code>on idle</code>	Componentes secundarios que deben descargarse sin interferir con la pintura crítica inicial del navegador.
<code>on timer(x)</code>	Retardo voluntario para dar prioridad a la descarga de recursos críticos del hilo principal.
<code>when condición</code>	Activación gobernada por reglas lógicas complejas (ej. permisos de usuario o finalización de procesos en lote).

Miniejercicio: Precarga Anticipada con Disparador Mixto

Enunciado: Configura un bloque diferido que precargue el fragmento JavaScript cuando el navegador entre en reposo (`prefetch on idle`) pero que solo renderice el componente visual cuando interactúes con el contenedor (`on interaction`).

Solución:

```
@defer (on interaction; prefetch on idle) {
  <app-consola-rendimiento />
} @placeholder {
  <button class="btn-consola">Abrir Consola de Rendimiento</button>
}
```

Listing 14: anexo3-mini-defer-prefetch-mixto.html

Explicación:

- **Optimización perceptiva:** La combinación de `prefetch on idle` con `on interaction` transfiere los recursos en segundo plano sin penalizar la interacción, logrando que el componente se monte de forma instantánea al pulsar el botón.

ANEXO IV: ¿Qué es un JSON?

En la ingeniería de software contemporánea, el intercambio de información entre aplicaciones cliente (navegadores web, dispositivos móviles) y servicios de datos (APIs de servidor, microservicios distribuidos) requiere un formato universal, ligero, agnóstico al lenguaje y comprensible por humanos y computadoras. Este anexo establece las bases técnicas y sintácticas del estándar JSON (*JavaScript Object Notation*).

1 Estructura Fundamental: Ficha de Entidad

Para comprender la arquitectura del formato, analiza la ficha técnica de un personaje explorador del universo de animación digital:

```
{
  "nombre": "Buzz Lightyear",
  "rango": "Guardián Espacial",
  "activo": true,
  "nivelEnergia": 98.5,
  "tripulacion": {
    "nave": "Star Cruiser 42",
    "capacidad": 2,
    "soporteVital": true
  },
  "misiones": [
    "Exploración Sector Gamma",
    "Defensa Planetaria",
    "Rescate en Cuadrante Zeta"
  ]
}
```

Listing 15: anexo4-p1-ficha-personaje.json

2 Tipos de Datos Soportados en JSON

El estándar JSON opera como un contenedor asociativo clave-valor (*key-value*) compuesto por tipos de datos fundamentales estrictamente delimitados:

- **Cadenas de Texto (Strings):** Delimitadas obligatoriamente por comillas dobles ("). El uso de comillas simples es inválido en la especificación. Admiten caracteres internacionales y Unicode (ej. "nombre": "Mickey Mouse").
- **Valores Numéricos (Numbers):** Representados sin delimitadores tipográficos, admitiendo enteros y decimales en punto flotante (ej. "nivelEnergia": 98.5).
- **Booleanos (Booleans):** Estados lógicos expresados exclusivamente en minúsculas: true o false. Escribir True con inicial mayúscula produce un fallo de parseo.
- **Objetos (Objects):** Conjuntos delimitados por llaves { } que agrupan sub-propiedades jerárquicas vinculadas semánticamente entre sí.
- **Arreglos o Listas (Arrays):** Colecciones ordenadas de elementos delimitadas por corchetes [], cuyos ítems contiguos se separan por comas.
- **Nulos (Null):** Representación formal de ausencia de valor mediante la palabra reservada null.

Miniejercicio: Detección y Enmienda de Infracciones de Sintaxis

Enunciado: Identifica y subsana los errores sintácticos del siguiente fragmento de datos para conformar un documento JSON estrictamente válido:

```
{
```

```
'personaje': 'Donald Duck',  
"rango": "Capitán",  
"horasVuelo": "120"  
"disponible": True  
}
```

Solución:

```
{  
  "personaje": "Donald Duck",  
  "rango": "Capitán",  
  "horasVuelo": 120,  
  "disponible": true  
}
```

Listing 16: anexo4-mini-personaje-valido.json

Explicación:

- **Reglas de conformidad:** Se sustituyen las comillas simples por dobles en la clave, se inserta la coma delimitadora entre propiedades, se retiran las comillas del valor numérico y se ajusta el booleano a minúsculas estrictas.

3 Serialización y Deserialización en Cliente

En el ecosistema ECMAScript moderno, la manipulación de cargas útiles JSON se articula mediante dos funciones del motor nativo:

- **JSON.stringify(objeto) (Serialización):** Transforma una estructura o modelo de objetos en memoria a una cadena alfanumérica transferible por protocolo HTTP.
- **JSON.parse(cadena) (Deserialización):** Convierte un flujo de texto recibido desde la red en un árbol de objetos navegable en memoria por TypeScript.

Serialización y Modelado de Flota Espacial en JSON

Enunciado

Construye una estructura JSON completa y formalmente válida que modele el registro operativo de un escuadrón de exploración. El objeto raíz debe incorporar: identificador del escuadrón, indicativo de misión activa, coordenadas de destino compuestas por un sub-objeto con latitud y longitud numéricas, y una lista de naves participantes con su comandante y nivel de autonomía.

Solución

```
{  
  "codigoEscuadron": "SQ-ALPHA-99",  
  "misionActiva": true,  
  "cuadrante": {  
    "sector": "Sector Estelar 04",  
    "coordenadaX": 142.85,  
    "coordenadaY": -89.40  
  },  
  "naves": [  

```

```
{
  "matricula": "SC-01",
  "comandante": "Buzz Lightyear",
  "autonomiaHoras": 48
},
{
  "matricula": "SC-02",
  "comandante": "Woody Pride",
  "autonomiaHoras": 36
}
]
```

Listing 17: anexo4-final-escuadron-espacial.json

Explicación

- **Estructura Jerárquica y Coherencia de Tipos:** El documento articula tipos primitivos (texto, numérico y booleano) con tipos compuestos (objeto anidado para la localización espacial y arreglo de objetos para el parque de naves), garantizando interoperabilidad inmediata con serializadores de cliente y servidor.

ANEXO V: Guía de Scaffolding en Angular

El *scaffolding* (andamiaje) en el desarrollo de software industrial alude a la generación automatizada de la arquitectura de carpetas, esquemas de configuración, contratos de tipado y esqueletos de código fuente. En el marco de Angular 19, el andamiaje asegura la uniformidad técnica del código, reduce drásticamente el código redundante (*boilerplate*) y acelera el arranque productivo en equipos multidisciplinares.

1 Anatomía del Árbol de Directorios Standalone

Al inicializar un espacio de trabajo moderno con el comando de CLI `ng new app-enterprise --routing --style scss`, se produce una estructura desacoplada sin archivos de módulo:

```
app-enterprise/
|-- .angular/           # Caché incremental de compilación del CLI
|-- src/
|   |-- app/
|   |   |-- app.component.html # Plantilla visual del contenedor raíz
|   |   |-- app.component.scss # Estilos encapsulados del componente raíz
|   |   |-- app.component.ts   # Componente standalone de arranque
|   |   |-- app.config.ts      # Proveedores globales (rutas, hidratación, HTTP)
|   |   |-- app.routes.ts      # Tabla declarativa de rutas de navegación
|   |-- assets/               # Recursos multimedia y estáticos
|   |-- index.html            # Documento contenedor de la Single Page Application
|   |-- main.ts                # Punto de entrada y bootstrap de la aplicación
|   |-- styles.scss            # Hoja de estilos transversal corporativa
|-- angular.json               # Manifiesto de configuración de compilación y builds
|-- package.json               # Dependencias NPM y scripts de ejecución
|-- tsconfig.json              # Opciones maestras del compilador TypeScript
```

2 Análisis de Ficheros Críticos de Inicialización

- **src/main.ts:** Punto inicial de ejecución. Invoca `bootstrapApplication(AppComponent, appConfig)`, sustituyendo al clásico arranque dinámico de módulos.
- **src/app/app.config.ts:** Centraliza la inyección de dependencias a nivel de aplicación mediante la interfaz `ApplicationConfig`, registrando utilidades como `provideRouter` o `provideHttpClient`.
- **angular.json:** Fichero de configuración que gobierna las metas de construcción (*targets*), perfiles de despliegue (*production, development*) y directrices de optimización.

3 Generación Automatizada de Elementos con Angular CLI

La herramienta de línea de comandos provee generadores esquemáticos para incorporar artefactos respetando las directrices de nomenclatura y encapsulación:

Miniejercicio: Scaffolding Rápido de Componente Funcional

Enunciado: Escribe la orden de CLI para generar un componente standalone denominado `visor-flota` dentro de la ruta modular `features/monitoreo`, omitiendo la generación de ficheros de prueba unitaria (`.spec.ts`).

Solución:

```
ng generate component features/monitoreo/visor-flota --skip-tests
```

Explicación:

- **Automatización granular:** El modificador `--skip-tests` evita la creación innecesaria del ar-

Artefacto Técnico	Comando CLI	Propósito en Arquitectura Frontend
Componente Standalone	<code>ng g c features/perfil</code>	Unidad gráfica interactiva con plantilla y estilos.
Servicio Inyectable	<code>ng g s core/services/api</code>	Gestión de estado de negocio y consumo HTTP global.
Guard Funcional	<code>ng g g core/guards/auth</code>	Control de acceso y protección perimetral de rutas.
Interceptor HTTP	<code>ng g interceptor core/jwt</code>	Manipulación transversal de cabeceras y telemetría de red.
Directiva Personalizada	<code>ng g d shared/directives/tooltip</code>	Extensión de comportamiento sobre nodos nativos del DOM.
Pipe de Transformación	<code>ng g p shared/pipes/moneda</code>	Filtro declarativo de formateo de datos en vista.

chivo de prueba cuando desarrollas prototipos rápidos o interfaces provisionales de visualización.

4 Buenas Prácticas de Estructuración Empresarial

Para evitar el desorden estructural en sistemas con decenas de vistas, se recomienda articular el código bajo el estándar de capas funcionales:

- **Capa /core:** Servicios globales singleton, interceptores de red y guardias de seguridad que se instancian una única vez.
- **Capa /shared:** Componentes de interfaz genéricos reutilizables (botones, tarjetas, modales), directivas cosméticas y pipes puros sin lógica de negocio propia.
- **Capa /features:** Módulos funcionales de negocio (ej. catálogo de atracciones, auditoría, panel administrativo), cada uno provisto de sus componentes de página y sus rutas hijas con carga perezosa.

Orquestación de Andamiaje para Arquitectura Enterprise

Enunciado

Plantea la secuencia completa de comandos de terminal para inicializar un proyecto empresarial denominado `galaxy-portal` configurado con SCSS, enrutador integrado y modo estricto de TypeScript. A continuación, genera el andamiaje base para un servicio central de autenticación y una característica de negocio para la gestión de personajes galácticos.

Solución

```
# 1. Inicialización del workspace standalone empresarial
ng new galaxy-portal --style scss --routing --strict

# 2. Acceso al entorno del proyecto
cd galaxy-portal

# 3. Creación de la capa core (servicio de sesión y guardia de rutas)
ng g service core/services/auth
ng g guard core/guards/auth --functional
```

```
# 4. Creación de la capa compartida (shared)
ng g component shared/components/navbar --standalone
ng g pipe shared/pipes/formato-identificador

# 5. Creación de la feature de negocio (personajes)
ng g component features/personajes/lista-personajes --standalone
ng g component features/personajes/detalle-personaje --standalone
```

Explicación

- **Modularidad y Escalabilidad:** La estructuración separa claramente los servicios de infraestructura global (*core*), los elementos estéticos transversales (*shared*) y las interfaces ligadas a dominios funcionales (*features*), permitiendo que la aplicación crezca sin colisiones arquitectónicas.

ANEXO VI: Arquitectura Standalone en Angular v19

La arquitectura Standalone constituye el estándar predeterminado y definitivo de desarrollo en Angular 19. Al erradicar los módulos organizativos tradicionales (`@NgModule`), el compilador trabaja sobre un grafo de dependencias directo a nivel de componente, lo que simplifica el modelo cognitivo del programador y reduce sensiblemente la sobrecarga de empaquetado.

1 Fundamentos del Paradigma Autónomo

Históricamente, los componentes requerían ser matriculados en la propiedad `declarations` de un `@NgModule`. Si el componente precisaba de elementos auxiliares, el módulo en su totalidad debía proveerlos en su lista de `imports`.

En Angular 19, cada componente adopta la filosofía *Component-First*:

- El componente declara explícitamente sus dependencias dentro de su propio decorador `@Component`.
- Se elimina por completo el archivo auxiliar `*.module.ts`.
- La carga perezosa de rutas se opera directamente con `loadComponent: () => import('./ruta').then(m => m.MiComponente)`.

2 Ventajas de Ingeniería y Productividad en el Desarrollo

La consolidación del modelo autónomo aporta beneficios directos en aplicaciones empresariales:

- **Enfoque Inmediato en la Lógica de Negocio:** Puedes dedicar tus esfuerzos a la implementación de estados reactivos con Signals y al ciclo de renderizado, sin invertir tiempo en mantener sincronizadas redes complejas de módulos intermedios.
- **Aislamiento y Reutilización:** Se fomenta la construcción de componentes atómicos de alta cohesión y acoplamiento nulo, susceptibles de ser transferidos e importados en cualquier vista del sistema de forma directa.
- **Tree-Shaking Avanzado:** Al conocer con precisión qué componentes y directivas son referenciados por cada nodo de la interfaz, el optimizador de producción suprime el código no consumido con máxima eficacia.

3 Anatomía Técnica de un Componente en Angular 19

A continuación se ilustra la definición de un componente autónomo en Angular 19. Observa la ausencia de dependencias de módulos obsoletos y el uso de reactividad por Signals:

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-heroe-profile',
  standalone: true,
  template: `
    <article class="perfil-card">
      <header>
        <h2>{{ personaje() }}</h2>
        <span class="badge-rol">{{ categoria() }}</span>
      </header>
      <p class="estado">Nivel de Operatividad: <strong>{{ energia() }}%</strong></p>
      <button (click)="incrementarEnergia()">Recargar Escudos</button>
    </article>
  `,
  styles: []
})
```

```

.perfil-card { background: #FFFFFF; border: 1px solid #CBD5E1; border-radius: 8px; padding: 1.25rem;
  ↳ max-width: 320px; }
header { display: flex; justify-content: space-between; align-items: center; border-bottom: 1px
  ↳ solid #E2E8F0; padding-bottom: 0.5rem; margin-bottom: 0.75rem; }
h2 { margin: 0; color: #1F2758; font-size: 1.15rem; }
.badge-rol { background: #F0FDF4; color: #059669; font-size: 0.75rem; padding: 2px 8px; border-
  ↳ radius: 4px; font-weight: bold; }
.estado { color: #475569; font-size: 0.9rem; margin-bottom: 1rem; }
button { background: #1E75B8; color: white; border: none; padding: 0.5rem 1rem; border-radius: 4px;
  ↳ cursor: pointer; font-weight: bold; width: 100%; }
`]
})
export class HeroeProfileComponent {
  personaje = signal<string>('Elsa de Arendelle');
  categoria = signal<string>('Realeza Mágica');
  energia = signal<number>(85);

  incrementarEnergia() {
    this.energia.update(valor => Math.min(100, valor + 5));
  }
}

```

Listing 18: anexo6-p3-heroe-profile.component.ts

Miniejercicio: Declaración de Selector Autónomo e Importación Local

Enunciado: Desarrolla un componente standalone denominado `BotonAccionComponent` que exponga un botón estilizado reutilizable, listo para ser consumido directamente por otros componentes sin intermediación de módulos.

Solución:

```

import { Component } from '@angular/core';

@Component({
  selector: 'app-boton-accion',
  standalone: true,
  template: `
    <button class="btn-corporativo"><ng-content></ng-content></button>
  `,
  styles: [`
    .btn-corporativo { background: #1E75B8; color: white; border: none; padding: 8px 16px; border
      ↳ -radius: 4px; font-weight: bold; cursor: pointer; }
  `]
})
export class BotonAccionComponent {}

```

Listing 19: anexo6-mini-boton-accion.component.ts

Explicación:

- **Reutilización Atómica:** El componente define `standalone: true` y un contenedor `<ng-content>` para proyección de contenido, permitiendo que cualquier otro componente autónomo lo importe directamente en su arreglo `imports`.

Panel de Expedición Galáctica con Arquitectura Standalone Pura

Enunciado

Desarrolla un componente standalone autosuficiente en Angular 19 que represente el panel de mando de una expedición galáctica. La clase debe gestionar el estado reactivo mediante Signals con el nombre del líder de misión (*Buzz Lightyear*) y una lista de coordenadas verificadas. La plantilla debe iterar dichas coordenadas utilizando el flujo de control integrado `@for` y actualizar el contador de cuadrantes de manera reactiva.

Solución

```
import { Component, signal, computed } from '@angular/core';

interface Cuadrante {
  id: number;
  nombre: string;
  explorado: boolean;
}

@Component({
  selector: 'app-panel-expedicion',
  standalone: true,
  template: `
    <div class="panel-expedicion">
      <header>
        <h3>Control de Vuelo: {{ liderMision() }}</h3>
        <span class="badge">Cuadrantes Totales: {{ totalCuadrantes() }}</span>
      </header>

      <section class="lista-sectores">
        @for (item of sectores(); track item.id) {
          <div class="fila-sector">
            <span>#{{ $index + 1 }} {{ item.nombre }}</span>
            <span [class]="item.explorado ? 'tag-ok' : 'tag-pending'">
              {{ item.explorado ? 'Explorado' : 'Pendiente' }}
            </span>
          </div>
        } @empty {
          <p class="sin-rutas">No se han registrado rutas de navegación.</p>
        }
      </section>

      <button (click)="registrarNuevoSector()">Añadir Coordenada</button>
    </div>
  `,
  styles: [`
    .panel-expedicion { background: #FFFFFF; border: 1px solid #CBD5E1; padding: 1.25rem; border-
      ↪ -radius: 8px; max-width: 420px; font-family: 'Noto Sans', sans-serif; }
    header { display: flex; justify-content: space-between; align-items: center; border-bottom:
      ↪ 2px solid #1E75B8; padding-bottom: 0.5rem; margin-bottom: 1rem; }
    h3 { margin: 0; color: #1F2758; font-size: 1.1rem; }
    .badge { background: #1F2758; color: white; padding: 2px 8px; border-radius: 12px; font-size
      ↪ : 0.75rem; }
    .fila-sector { display: flex; justify-content: space-between; padding: 0.4rem 0; border-`

```

```
↪ bottom: 1px solid #F1F5F9; font-size: 0.85rem; }
.tag-ok { color: #059669; font-weight: bold; }
.tag-pending { color: #D97706; font-weight: bold; }
.sin-rutas { color: #64748B; font-style: italic; text-align: center; }
button { margin-top: 1rem; background: #1E75B8; color: white; border: none; padding: 0.5rem
↪ 1rem; border-radius: 4px; cursor: pointer; width: 100%; font-weight: bold; }
`]
})
export class PanelExpedicionComponent {
  liderMision = signal<string>('Buzz Lightyear');
  sectores = signal<Cuadrante[]>([
    { id: 1, nombre: 'Nebulosa Orión Alfa', explorado: true },
    { id: 2, nombre: 'Cinturón de Asteroides Zeta', explorado: false }
  ]);

  totalCuadrantes = computed(() => this.sectores().length);

  registrarNuevoSector() {
    const nuevoId = this.sectores().length + 1;
    const nuevo: Cuadrante = {
      id: nuevoId,
      nombre: `Sector Profundo 0${nuevoId}`,
      explorado: false
    };
    this.sectores.update(lista => [...lista, nuevo]);
  }
}
```

Listing 20: anexo6-final-panel-expedicion.component.ts

Explicación

- **Arquitectura Standalone y Reactividad Pura:** El componente es completamente autónomo, carece de acoplamiento a módulos y gestiona su estado mediante Signals mutables (`signal()`) y computadas (`computed()`), sincronizándose dinámicamente con el flujo de compilador `@for`.